API: ESERCITAZIONI

LOGICHE

Gabriele Paveri-Fontana

Mail: gabriele.paveri@polimi.it

Recap Logiche & Linguaggi

- Nell'ambito dei linguaggi formali, possiamo utilizzare particolari formalismi logici come strumenti con cui definire linguaggi: il linguaggio definito da una formula è l'insieme delle parole per le quali la formula è vera.
- Vedremo due esempi di tali formalismi:
  - MFO: Monadic First-order Logic
  - MSO: Monadic Second-order logic
- Vedremo infine come si possono specificare le pre- e post-condizioni di un programma tramite formule logiche al prim'ordine.

MFO: Monadic First-order Logic

Recap MFO: sintassi

Fissato un alfabeto I e un insieme di variabili V, le MFO-formule sono date da:

$$\varphi := a(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \forall x\, \varphi, \qquad \text{con } x, y \in V, a \in I$$

Si tratta cioè della logica al prim'ordine ottenuta, fissato un alfabeto I, a partire dal linguaggio $(V, \mathcal{F}, \mathcal{P})$ dato da:

- V insieme di variabili (noi useremo $x, y, z, \ldots$) disgiunto da I
- $\mathcal{F} = \emptyset$, cioè nessun simbolo di funzione (e dunque nessuna costante)
- $\mathcal{P} = \{a | a \in I\} \cup \{<\}$ insieme dei predicati:
  - 'a' pred. unario per ogni $a \in I$
  - '$<$' predicato binario

Recap MFO: semantica

Le MFO-formule vengono interpretate su parole di I^*.

- Il valore di verità di una MFO-formula φ rispetto a una parola $w \in I^*$ e ad una valutazione $v: V \to \{0, \dots, |w| - 1\}$ è definito induttivamente da:
  - $w, v \vDash a(x)$ sse $w\big(v(x)\big) = a$
  - $w, v \vDash x < y$ sse $v(x) < v(y)$
  - $w, v \vDash \neg\varphi_1$ sse $w, v \nvDash \varphi_1$
  - $w, v \vDash \varphi_1 \wedge \varphi_2$ sse $w, v \vDash \varphi_1$ ed $w, v \vDash \varphi_2$
  - $w, v \vDash \forall x\, \varphi_1$ sse $w, v[t/x] \vDash \varphi_1$ per ogni $0 \leq t \leq |w| - 1$

cioè in w, v è vero il predicato $a(x)$ sse il $v(x)$-esimo simbolo di w è a. N.B.: primo simbolo di w ha indice 0.

- Scriviamo $w, v \vDash \varphi$ per indicare che φ è **vera** rispetto a w, v

  Scriviamo $w, v \nvDash \varphi$ per indicare che φ è **falsa** rispetto a w, v

Recap MFO: Linguaggio definito

- Una parola $w \in I^*$ è detta **modello** di φ (si scrive $w \vDash \varphi$) se per ogni valutazione v si ha $w, v \vDash \varphi$
- Una MFO-formula φ è **chiusa** se non contiene occorrenze di variabili libere, ovvero se tutte le variabili che compaiono in φ sono quantificate
- N.B.: il valore di verità di una formula chiusa φ non dipende dalla valutazione v
- Il linguaggio definito da una *MFO-formula chiusa* φ è il sottoinsieme $L(\varphi)$ di I^* dato da tutti i modelli di φ (ovvero tutte le parole rispetto a cui φ è vera):

$$L(\varphi) = \{w \in I^* \mid w \vDash \varphi\}$$

Recap MFO: Potenza espressiva

- Un linguaggio L è definibile (esprimibile) in MFO se esiste una MFO-formula chiusa φ tale che $L(\varphi) = L$
- La famiglia dei linguaggi MFO-definibili è denotata con $\mathcal{L}_{\text{MFO}}$
- MFO è un formalismo strettamente meno potente degli FSA, ovvero:

$$\mathcal{L}_{\text{MFO}} \subsetneq \mathcal{RL}$$

Recap MFO: Abbreviazioni classiche

- Introduciamo le seguenti abbreviazioni, che sono classicamente utilizzate in logica al prim'ordine (FOL):

  - $\varphi_1 \vee \varphi_2 \quad :\equiv \quad \neg(\neg\varphi_1 \wedge \neg\varphi_2)$
  - $\varphi_1 \rightarrow \varphi_2 \quad :\equiv \quad \neg\varphi_1 \vee \varphi_2$
  - $\exists x\, \varphi \quad :\equiv \quad \neg\forall x(\neg\varphi)$
  - $x = y \quad :\equiv \quad \neg(x < y) \wedge \neg(y < x)$
  - $x \leq y \quad :\equiv \quad \neg(y < x)$

Recap MFO: Altre abbreviazioni

- Introduciamo ora altre abbreviazioni, specificamente pensate per MFO:
  - $x = 0 \quad :\equiv \quad \forall y(\neg(y < x))$
  - $y = S(x) \quad :\equiv \quad x < y \wedge \neg\exists z(x < z \wedge z < y)$
  - $x = k \quad :\equiv \quad \exists y_0, \dots, y_{k-1}(y_0 = 0 \wedge y_1 = S(y_0) \wedge \cdots \wedge x = S(y_{k-1}))$
  - $y = x + 1 \quad :\equiv \quad y = S(x)$
  - $y = x + k \quad :\equiv \quad \exists z_1, \dots, z_{k-1}(z_1 = x + 1 \wedge z_2 = z_1 + 1 \wedge \cdots \wedge y = z_{k-1} + 1$
  - $y = x - 1 \quad :\equiv \quad x = S(y)$
  - $y = x - k \quad :\equiv \quad x = y + k$
  - $last(x) \quad :\equiv \quad \neg\exists y(y > x)$

N.B.: Utilizzeremo anche altre abbreviazioni, la cui definizione sarà ovvia a partire da queste.

Recap MFO: Cosa possiamo chiederci?

- Data una MFO-formula φ, descrivere il linguaggio $L(\varphi)$ definito da φ
- Dato un linguaggio L (che stia in $\mathcal{L}_{\text{MFO}}$), esibire una MFO-formula φ che definisca L, ovvero tale che che $L(\varphi) = L$

Es.1 Data la formula φ, descrivere $L(\varphi)$

$$\varphi := \forall x \Big(\big(x = 0 \wedge a(x) \big) \rightarrow \big(\exists y \big(last(y) \wedge a(y) \big) \big) \Big)$$

Es.1

$$\varphi := \forall x \Big(\big(x = 0 \wedge a(x) \big) \rightarrow \big(\exists y \big(last(y) \wedge a(y) \big) \big) \Big)$$

$$L = \{ w \in I^* \mid \text{se } w \text{ inizia con } 'a', \text{allora } w \text{ termina con } 'a' \}$$

Es.2 Data la formula φ, descrivere $L(\varphi)$

$$\varphi \coloneqq \exists x \Big(x = 0 \wedge \forall y \big(y = S(x) \rightarrow last(y) \big) \Big)$$

Es.2

$$\varphi := \exists x \left(x = 0 \wedge \forall y \left(y = S(x) \rightarrow last(y) \right) \right)$$

$$L = \{ w \in I^* \mid |w| = 1 \text{ oppure } |w| = 2 \}$$

Es.3 Data la formula φ, descrivere $L(\varphi)$

$$\varphi := \forall x\ last(x)$$

Es.3

$$\varphi := \forall x\ last(x)$$

$$L = \{w \in I^* |\ |w| = 1 \text{ oppure } w = \epsilon\}$$

Es.4 Esibire φ MFO che definisca L

$L = \{\epsilon\}$

$L = I^* \setminus \{\epsilon\}$

Es.4

$$L = \{\epsilon\}$$

$$\varphi \coloneqq \neg\exists x \left(a(x) \vee \neg a(x)\right)$$

oppure

$$\varphi \coloneqq \forall x \left(a(x) \wedge \neg a(x)\right)$$

$$L = I^* \setminus \{\epsilon\}$$

$$\varphi \coloneqq \exists x \left(a(x) \vee \neg a(x)\right)$$

oppure

$$\varphi \coloneqq \neg\forall x \left(a(x) \wedge \neg a(x)\right)$$

Es.5 Esibire φ MFO che definisca L

$L = \emptyset$

$L = I^*$

Es.5

$$L = \emptyset$$

$$\varphi \coloneqq \exists x \left(a(x) \wedge \neg a(x)\right)$$

oppure

$$\varphi \coloneqq \neg \forall x \left(a(x) \vee \neg a(x)\right)$$

$$L = I^*$$

$$\varphi \coloneqq \neg \exists x \left(a(x) \wedge \neg a(x)\right)$$

oppure

$$\varphi \coloneqq \forall x \left(a(x) \vee \neg a(x)\right)$$

Es.6 Esibire φ MFO che definisca L

$$L = \{w \in I^* \mid w \text{ inizia con } 'a'\}$$

Es.6

$$L = \{w \in I^* \mid w \text{ inizia con } 'a'\}$$

$$\varphi \coloneqq \exists x \left(x = 0 \wedge a(x)\right)$$

Es.7 Esibire φ MFO che definisca L

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ogni 'a' sia preceduta da una 'b' e tra di esse ci siano solo 'c', in qualunque numero.

Es.7

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ogni 'a' sia preceduta da una 'b' e tra di esse ci siano solo 'c', in qualunque numero.

$$\varphi \coloneqq \forall x \left(a(x) \rightarrow \exists y \left(y < x \wedge b(y) \wedge \forall z \left(y < z < x \rightarrow c(z) \right) \right) \right)$$

E' un'abbreviazione per:

$$y < z \wedge z < x$$

Es.8 Esibire φ MFO che definisca L

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ogni 'a' sia preceduta da una 'b' in una delle 3 posizioni precedenti.

Es.8

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ogni 'a' sia preceduta da una 'b' in una delle 3 posizioni precedenti.

$$\varphi \coloneqq \forall x \left(a(x) \rightarrow \exists y (b(y) \wedge x - 3 \leq y \leq x - 1)\right)$$

E' un'abbreviazione per:

$y = x - 3 \vee y = x - 2 \vee y = x - 1$

Es.9 Esibire φ MFO che definisca L

Sia I un alfabeto contenente i simboli 'a' e 'b', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che tra ogni coppia di 'a' ci sia almeno una 'b'.

N.B.: Si intende due 'a' qualunque all'interno della parola: potrebbero esserci altre 'a' tra la prima e la seconda!

Es.9

Sia I un alfabeto contenente i simboli 'a' e 'b', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che tra ogni coppia di 'a' ci sia almeno una 'b'.

$$\varphi \coloneqq \forall x, y \left((a(x) \wedge a(y) \wedge x < y) \rightarrow \exists z \big(x < z < y \wedge b(z) \big) \right)$$

Es.10 Esibire φ MFO che definisca L

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola contiene al più una volta i simboli 'a', 'b', 'c'.

Es.10

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola contiene al più una volta i simboli 'a', 'b', 'c'.

Idea 1

E' un'abbreviazione per:
$\neg(x = y)$

$$\varphi := \forall x \left(\begin{array}{l} a(x) \rightarrow \forall y\big(x \neq y \rightarrow \neg a(y)\big) \\ \wedge \\ b(x) \rightarrow \forall y\big(x \neq y \rightarrow \neg b(y)\big) \\ \wedge \\ c(x) \rightarrow \forall y\big(x \neq y \rightarrow \neg c(y)\big) \end{array} \right)$$

Es.10

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola contiene al più una volta i simboli 'a', 'b', 'c'.

Idea 2

$$\varphi := \quad \forall x, y \left\langle \begin{array}{c} \big(a(x) \wedge a(y)\big) \rightarrow x = y \\ \wedge \\ \big(b(x) \wedge b(y)\big) \rightarrow x = y \\ \wedge \\ \big(c(x) \wedge c(y)\big) \rightarrow x = y \end{array} \right\rangle$$

Es.10

Sia I un alfabeto contenente i simboli 'a', 'b', 'c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola contiene al più una volta i simboli 'a', 'b', 'c'.

Idea 3

$$\varphi := \forall x, y \left\langle \begin{array}{c} x \neq y \rightarrow \neg\big(a(x) \wedge a(y)\big) \\ \wedge \\ x \neq y \rightarrow \neg\big(b(x) \wedge b(y)\big) \\ \wedge \\ x \neq y \rightarrow \neg\big(c(x) \wedge c(y)\big) \end{array} \right\rangle$$

Es.11 Esibire φ MFO che definisca L

Sia I un alfabeto contenente il simbolo 'a', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ogni 10 simboli vi possano essere al più due 'a'.

Es.11

Sia I un alfabeto contenente il simbolo 'a', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ogni 10 simboli vi possano essere al più due 'a'.

$$\varphi := \quad \forall x, y \left((y = x + 9) \rightarrow \neg \exists z_1, z_2, z_3 \left\langle \begin{array}{c} (x \leq z_1 < z_2 < z_3 \leq y) \\ \wedge \\ a(z_1) \wedge a(z_2) \wedge a(z_3) \end{array} \right\rangle \quad \right)$$

Es.12 Esibire φ MFO che definisca L

Sia I un alfabeto contenente il simbolo 'a', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che le 'a' possano comparire solo in sequenze di 4.

Es.12

Sia I un alfabeto contenente il simbolo 'a', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che le 'a' possano comparire solo in sequenze di 4.

$$\varphi := \forall x \left(\left(a(x) \wedge \left(x = 0 \vee \exists z \left(z = x - 1 \wedge \neg a(z) \right) \right) \right) \rightarrow \left\langle \begin{array}{c} \exists y_1 \left(y_1 = x + 1 \wedge a(y_1) \right) \\ \wedge \\ \exists y_2 \left(y_2 = x + 2 \wedge a(y_2) \right) \\ \wedge \\ \exists y_3 \left(y_3 = x + 3 \wedge a(y_3) \right) \\ \wedge \\ \exists y_4 \left(y_4 = x + 4 \wedge \neg a(y_4) \right) \end{array} \right\rangle \right)$$

Es.13 Esibire φ MFO che definisca L

Sia I un alfabeto contenente i simboli 'a' e 'b', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

- le 'a' compaiano in tutte e sole le posizioni $3k, k \geq 0$
- tra due 'a' consecutive vi sia sempre esattamente una 'b'
- l'ultima lettera sia una 'a'

Es.13

Sia I un alfabeto contenente i simboli 'a' e 'b', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

- le 'a' compaiano in tutte e sole le posizioni $3k, k \geq 0$
- tra due 'a' consecutive vi sia sempre esattamente una 'b'
- l'ultima lettera sia una 'a'

$$\varphi := \varphi_1 \wedge \varphi_2 \wedge \varphi_3$$

Es.13

Sia I un alfabeto contenente i simboli 'a' e 'b', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

- le 'a' compaiano in tutte e sole le posizioni $3k, k \geq 0$
- tra due 'a' consecutive vi sia sempre esattamente una 'b'
- l'ultima lettera sia una 'a'

$$\varphi \coloneqq \varphi_1 \wedge \varphi_2 \wedge \varphi_3$$

$$\varphi_1 \coloneqq \exists x \left(x = 0 \wedge a(x)\right)$$

$$\varphi_2 \coloneqq \forall x \left(a(x) \rightarrow \left(last(x) \vee \exists y \left(y = x + 3 \wedge a(y) \wedge \forall z \left(x < z < y \rightarrow \neg a(z)\right)\right)\right)\right)$$

Es.13

Sia I un alfabeto contenente i simboli 'a' e 'b', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

- le 'a' compaiano in tutte e sole le posizioni $3k, k \geq 0$
- tra due 'a' consecutive vi sia sempre esattamente una 'b'
- l'ultima lettera sia una 'a'

$$\varphi := \varphi_1 \wedge \varphi_2 \wedge \varphi_3$$

$$\varphi_3 := \forall x, y \left(\begin{array}{c} x < y \wedge a(x) \wedge a(y) \wedge \forall z \big(x < z < y \rightarrow \neg a(z) \big) \\ \downarrow \\ \exists u \Big(x < u < y \wedge b(u) \wedge \forall v \big(x < v < y \wedge v \neq u \rightarrow \neg b(v) \big) \Big) \end{array} \right)$$

MSO: Monadic Second-order Logic

Recap MSO: sintassi

Fissato un alfabeto I e due insieme di variabili V_1, V_2, le MSO-formule sono date da:

$$\varphi \coloneqq a(x) \mid P(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \forall x\, \varphi \mid \forall P\, \varphi,$$
$$\text{con } x, y \in V_1, P \in V_2, a \in I$$

Si tratta cioè della logica al second'ordine ottenuta, fissato un alfabeto I, a partire dal linguaggio $(V_1, V_2, \mathcal{F}, \mathcal{P})$ dato da:

- V_1 insieme di variabili del prim'ordine $(x, y, z, \ldots)$ disgiunto da I
- V_2 insieme di variabili del second'ordine $(P, Q, \ldots)$ disgiunto da I e da V_1
- $\mathcal{F} = \emptyset$
- $\mathcal{P} = \{a | a \in I\} \cup \{P | P \in V_2\} \cup \{<\}$

Recap MSO: semantica

Le MSO-formule vengono interpretate su parole di I^*.

- Il valore di verità di una MSO-formula φ rispetto a una parola $w \in I^*$ e ad una coppia di valutazioni $v_1 : V_1 \to \{0, \dots, |w| - 1\}$, $v_2 : V_2 \to \wp(\{0, \dots, |w| - 1\})$ è definito induttivamente da:
  - $w, v_1, v_2 \vDash a(x), x < y, \neg\varphi_1, \varphi_1 \wedge \varphi_2, \forall x\, \varphi_1$ come per MFO
  - $w, v_1, v_2 \vDash P(x)$ sse $v_1(x) \in v_2(P)$
  - $w, v_1, v_2 \vDash \forall P\, \varphi$ sse $w, v_1, v_2[S/P] \vDash \varphi$ per ogni $S \subseteq \{0, \dots, |w| - 1\}$

- $w, v \vDash \varphi$ → φ è **vera** rispetto a w, v_1, v_2
  $w, v \nvDash \varphi$ → φ è **falsa** rispetto a w, v_1, v_2

cioè in w, v_1, v_2 è vero il predicato $P(x)$ sse la $v_1(x)$-esima posizione di w appartiene a $v_2(P)$.

Recap MSO: Linguaggio definito

- Una parola $w \in I^*$ è detta **modello** di φ (si scrive $w \vDash \varphi$) se per ogni v_1, v_2 valutazioni si ha $w, v_1, v_2 \vDash \varphi$
- Una MSO-formula φ è **chiusa** se non contiene occorrenze di variabili libere, ovvero se tutte le variabili che compaiono in φ sono quantificate, sia quelle del primo sia quelle del second'ordine
- N.B.: il valore di verità di una formula chiusa φ non dipende dalle valutazioni v_1, v_2
- Il linguaggio definito da una *MFO-formula <u>chiusa</u>* φ è il sottoinsieme $L(\varphi)$ di I^* dato da tutti i modelli di φ (ovvero tutte le parole rispetto a cui φ è vera):

$$L(\varphi) = \{w \in I^* \mid w \vDash \varphi\}$$

Recap MSO: Potenza espressiva

- Un linguaggio L è definibile (esprimibile) in MSO se esiste una MSO-formula chiusa φ tale che $L(\varphi) = L$
- La famiglia dei linguaggi MSO-definibili è denotata con $\mathcal{L}_{\text{MSO}}$
- MSO è un formalismo equipotente agli FSA, ovvero:

$$\mathcal{L}_{\text{MSO}} = \mathcal{RL}$$

Recap MSO: Abbreviazioni ulteriori

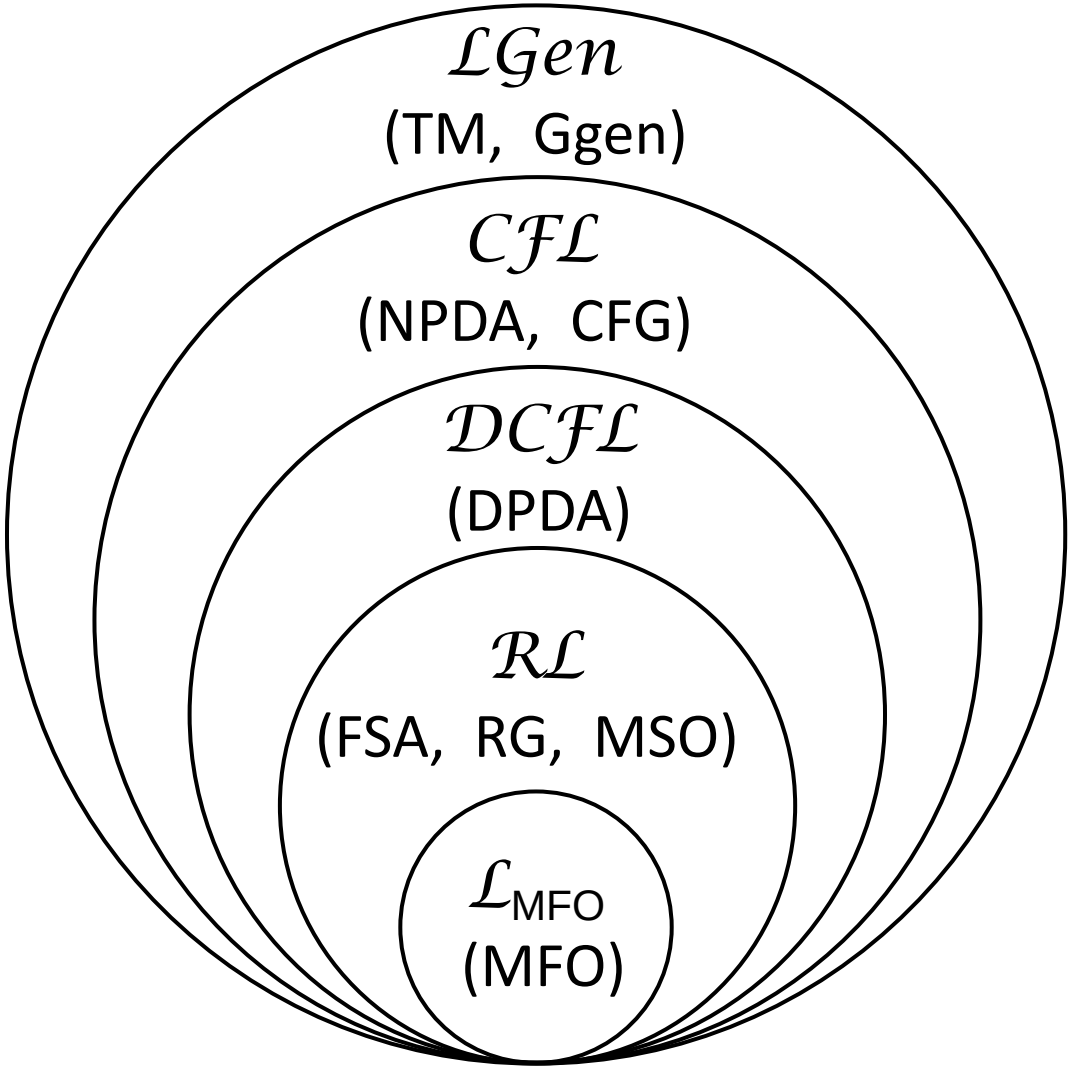
- Oltre a tutte le abbreviazioni introdotte per MFO, che rimangono valide, aggiungiamo le seguenti:

$$
\begin{aligned}
&- \quad x \in P && \coloneqq && P(x) \\
&- \quad P \subseteq Q && \coloneqq && \forall x\,(x \in P \rightarrow x \in Q) \\
&- \quad P = Q && \coloneqq && (P \subseteq Q) \wedge (Q \subseteq P) \\
&- \quad P \neq Q && \coloneqq && \neg(P = Q)
\end{aligned}
$$

Recap MSO: Cosa possiamo chiederci?

- Data una MSO-formula φ, descrivere il linguaggio $L(\varphi)$ definito da φ
- Dato un linguaggio regolare L, esibire una MSO-formula φ che definisca L, ovvero tale che che $L(\varphi) = L$

Recap Quadro generale completo



Es.14 Data la formula φ, descrivere $L(\varphi)$

$$\varphi \coloneqq \forall x, y \left(\begin{array}{c} \big(x < y \wedge a(x) \wedge a(y) \wedge \forall z \big(x < z < y \rightarrow \neg a(z)\big)\big) \\ \downarrow \\ \exists P \Big(x \in P \wedge y \in P \wedge \forall z, t \Big((x \leq z < y \wedge t = z + 1) \rightarrow \big(z \in P \leftrightarrow \neg (t \in P)\big)\Big)\Big) \end{array} \right)$$

Es.14

$$\varphi := \forall x, y \left(\begin{array}{c} \Big(x < y \wedge a(x) \wedge a(y) \wedge \forall z\big(x < z < y \rightarrow \neg a(z)\big)\Big) \\ \downarrow \\ \exists P \Big(x \in P \wedge y \in P \wedge \forall z, t \Big((x \leq z < y \wedge t = z + 1) \rightarrow \big(z \in P \leftrightarrow \neg(t \in P)\big)\Big)\Big) \end{array} \right)$$

<u>Osservazione</u>: $\forall x, y$ si ha che $P = P_{x,y}$ è tale che

- $\{z = x + 2k | k \geq 0, z \leq y\} \subseteq P_{x,y}$
- $\{z = x + 2k + 1 | k \geq 0, z < y\} \subseteq \neg P_{x,y}$

Es.14

$$\varphi := \forall x, y \left(\begin{array}{c} \big(x < y \wedge a(x) \wedge a(y) \wedge \forall z \big(x < z < y \rightarrow \neg a(z)\big)\big) \\ \downarrow \\ \exists P \Big(x \in P \wedge y \in P \wedge \forall z, t \Big((x \leq z < y \wedge t = z + 1) \rightarrow \big(z \in P \leftrightarrow \neg (t \in P)\big)\Big)\Big) \end{array} \right)$$

$L(\varphi) \subseteq I^*$, con $a \in I$, è il linguaggio ogni cui parola è tale che tra due 'a' consecutive vi sia un numero dispari di simboli.

Es.15 Esibire φ MSO che definisca L

Sia I un alfabeto contenente i simboli ‘a’,‘b’,‘c’, e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

1. ogni ‘b’ compaia tra una ‘a’ ed una ‘c’
2. tra una ‘a’ ed una ‘c’ compaia al massimo una ‘b’
3. il numero di ‘b’ sia multiplo di 3 $(\#b = 3k, k \geq 0)$

Es.15

Sia I un alfabeto contenente i simboli ‘a’,‘b’,‘c’, e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

1. ogni ‘b’ compaia tra una ‘a’ ed una ‘c’
2. tra una ‘a’ ed una ‘c’ compaia al massimo una ‘b’
3. il numero di ‘b’ sia multiplo di 3 $(\#b = 3k, k \geq 0)$

$$\varphi := \varphi_1 \wedge \varphi_2 \wedge \varphi_3$$

Es.15

Sia I un alfabeto contenente i simboli 'a','b','c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

1. ogni 'b' compaia tra una 'a' ed una 'c'
2. tra una 'a' ed una 'c' compaia al massimo una 'b'
3. il numero di 'b' sia multiplo di 3 $(\#b = 3k, k \geq 0)$

$$\varphi := \varphi_1 \wedge \varphi_2 \wedge \varphi_3$$

$$\varphi_1 := \forall x \left(b(x) \rightarrow \exists u, v \big(u < x < v \wedge a(u) \wedge c(v) \big) \right)$$

Es.15

Sia I un alfabeto contenente i simboli 'a','b','c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

1. ogni 'b' compaia tra una 'a' ed una 'c'
2. tra una 'a' ed una 'c' compaia al massimo una 'b'
3. il numero di 'b' sia multiplo di 3 $(\#b = 3k, k \geq 0)$

$$\varphi := \varphi_1 \wedge \varphi_2 \wedge \varphi_3$$

$$\varphi_2 := \forall x, y\ (x < y \wedge a(x) \wedge c(y) \rightarrow \left\langle \begin{array}{c} \exists u \left(x < u < y \wedge b(u) \wedge \forall v \left(x < v < y \wedge v \neq u \rightarrow \neg b(v)\right)\right) \\ \vee \\ \forall u \left(x < u < y \rightarrow \neg b(u)\right) \end{array} \right\rangle)$$

Es.15

Sia I un alfabeto contenente i simboli 'a','b','c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

1. ogni 'b' compaia tra una 'a' ed una 'c'
2. tra una 'a' ed una 'c' compaia al massimo una 'b'
3. il numero di 'b' sia multiplo di 3 $(\#b = 3k, k \geq 0)$

Idea per φ_3:

- devono esistere P_0, P_1, P_2 t.c.
  - $x \in P_0 \Leftrightarrow$ il numero di 'b' fino alla posizione x è $3k$
  - $x \in P_1 \Leftrightarrow$ il numero di 'b' fino alla posizione x è $3k+1$
  - $x \in P_2 \Leftrightarrow$ il numero di 'b' fino alla posizione x è $3k+2$
- l'ultima posizione deve appartenere a P_0

Es.15

Sia I un alfabeto contenente i simboli 'a','b','c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

1. ogni 'b' compaia tra una 'a' ed una 'c'
2. tra una 'a' ed una 'c' compaia al massimo una 'b'
3. il numero di 'b' sia multiplo di 3 ($\#b = 3k, k \geq 0$)

$$\varphi_3 \coloneqq \exists P_0, P_1, P_2 \; \forall x (\psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4)$$

$$\psi_1 \coloneqq \neg\big(P_0(x) \wedge P_1(x)\big) \wedge \neg\big(P_0(x) \wedge P_2(x)\big) \wedge \neg\big(P_1(x) \wedge P_2(x)\big)$$

$$\psi_2 \coloneqq \big(x = 0 \wedge b(x) \rightarrow P_1(x)\big) \wedge \big(x = 0 \wedge \neg b(x) \rightarrow P_0(x)\big)$$

$$\psi_3 \coloneqq last(x) \rightarrow P_0(x)$$

Es.15

Sia I un alfabeto contenente i simboli 'a','b','c', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che:

1. ogni 'b' compaia tra una 'a' ed una 'c'
2. tra una 'a' ed una 'c' compaia al massimo una 'b'
3. il numero di 'b' sia multiplo di 3 ($\#b = 3k, k \geq 0$)

$$\varphi_3 \coloneqq \exists P_0, P_1, P_2 \, \forall x (\psi_1 \wedge \psi_2 \wedge \psi_3 \wedge \psi_4)$$

$$\psi_4 \coloneqq \forall y (y = x + 1 \rightarrow \left(\begin{array}{c} \Big(\big((P_0(x) \wedge b(y)) \rightarrow P_1(y) \big) \wedge \big((P_0(x) \wedge \neg b(y)) \rightarrow P_0(y) \big) \Big) \\ \wedge \\ \Big(\big((P_1(x) \wedge b(y)) \rightarrow P_2(y) \big) \wedge \big((P_1(x) \wedge \neg b(y)) \rightarrow P_1(y) \big) \Big) \\ \wedge \\ \Big(\big((P_2(x) \wedge b(y)) \rightarrow P_0(y) \big) \wedge \big((P_2(x) \wedge \neg b(y)) \rightarrow P_2(y) \big) \Big) \end{array} \right))$$

Es.16 Esibire φ MSO che definisca L

Sia I un alfabeto contenente il simbolo 'a', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ci siano almeno due 'a' e che tra ogni coppia di 'a' vi siano esattamente $4k$, con $k > 0$, simboli.

Es.16

Sia I un alfabeto contenente il simbolo 'a', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ci siano almeno due 'a' e che tra ogni coppia di 'a' vi siano esattamente $4k$, con $k > 0$, simboli.

Idea:

- $\forall x \neq y$ t.c. $a(x)$ e $a(y)$, devono esistere P_1, P_2, P_3, P_4 t.c.

$$\{z = x + 1 + 4k | k \geq 0, z < y\} \subseteq P_1$$
$$\{z = x + 2 + 4k | k \geq 0, z < y\} \subseteq P_2$$
$$\{z = x + 3 + 4k | k \geq 0, z < y\} \subseteq P_3$$
$$\{z = x + 4 + 4k | k \geq 0, z < y\} \subseteq P_4$$

$$P_i \cap P_j = \emptyset$$
$$\forall i \neq j \in \{1,2,3,4\}$$

- l'ultima posizione prima di y deve appartenere a P_4

Es.16

Sia I un alfabeto contenente il simbolo 'a', e sia $L \subseteq I^*$ il linguaggio ogni cui parola è tale che ci siano almeno due 'a' e che tra ogni coppia di 'a' vi siano esattamente $4k$, con $k > 0$, simboli.

$$\varphi \coloneqq \exists x, y\ \varphi_1 \wedge \forall x, y\ \Big(\varphi_1 \rightarrow \big(\exists z(x < z < y) \wedge \exists P_1, P_2, P_3, P_4 \forall z(\varphi_2 \wedge \varphi_3)\big)\Big)$$

$$\varphi_1 \coloneqq x < y \wedge a(x) \wedge a(y)$$

$$\varphi_2 \coloneqq \neg\big(P_1(z) \wedge P_2(z)\big) \wedge \neg\big(P_1(z) \wedge P_3(z)\big) \wedge \neg\big(P_1(z) \wedge P_4(z)\big) \wedge \\ \wedge \neg\big(P_2(z) \wedge P_3(z)\big) \wedge \neg\big(P_2(z) \wedge P_4(z)\big) \wedge \neg\big(P_3(z) \wedge P_4(z)\big)$$

$$\varphi_3 \coloneqq x < z < y \rightarrow \begin{pmatrix} \big(z = x + 1 \rightarrow P_1(z)\big) \wedge \big(z = y - 1 \rightarrow P_4(z)\big) \wedge \\ \wedge \big(P_1(z) \leftrightarrow P_2(z+1)\big) \wedge \big(P_2(z) \leftrightarrow P_3(z+1)\big) \wedge \\ \wedge \big(P_3(z) \leftrightarrow P_4(z+1)\big) \wedge \big(P_4(z) \leftrightarrow P_1(z+1)\big) \end{pmatrix}$$

Es.17 Esibire φ MSO che definisca L

$$L = \{aa, bb\}^+$$

Es.17

$$L = \{aa, bb\}^+$$

Idea:

- deve esistere P tale che

$$P = \{x = 4k | k \geq 0\} \cup \{x = 4k + 1 | k \geq 0\}$$

$$\neg P = \{x = 4k + 2 | k \geq 0\} \cup \{x = 4k + 3 | k \geq 0\}$$

- posizioni contigue che appartengono a P (risp. $\neg P$) devono contenere lo stesso carattere

Es.17

$$L = \{aa, bb\}^+$$

$$\varphi := \forall x \left(a(x) \vee b(x)\right) \wedge \exists P(\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4 \wedge \varphi_5)$$

$$\varphi_1 := \exists x, y\ (x = 0 \wedge y = 1 \wedge P(x) \wedge P(y))$$

$$\varphi_2 := \forall x, y \left(\left(y = x + 1 \wedge P(x) \wedge P(y)\right) \rightarrow \left(last(y) \vee \exists u, v\left(u = y + 1 \wedge v = y + 2 \wedge \neg P(u) \wedge \neg P(v)\right)\right)\right)$$

$$\varphi_3 := \forall x, y \left(\left(y = x + 1 \wedge \neg P(x) \wedge \neg P(y)\right) \rightarrow \left(last(y) \vee \exists u, v\left(u = y + 1 \wedge v = y + 2 \wedge P(u) \wedge P(v)\right)\right)\right)$$

$$\varphi_4 := \forall x, y \left(\left(y = x + 1 \wedge P(x) \wedge P(y)\right) \rightarrow \left(\left(a(x) \leftrightarrow a(y)\right) \wedge \left(b(x) \leftrightarrow b(y)\right)\right)\right)$$

$$\varphi_5 := \forall x, y \left(\left(y = x + 1 \wedge \neg P(x) \wedge \neg P(y)\right) \rightarrow \left(\left(a(x) \leftrightarrow a(y)\right) \wedge \left(b(x) \leftrightarrow b(y)\right)\right)\right)$$

Pre- e post-condizioni di programmi

Recap Pre- e post-condizioni di programmi

- Possiamo utilizzare formule logiche (FOL: first order logic) anche per definire pre e post-condizioni di programmi, ovvero per fornire specifiche sul loro comportamento desiderato:

  - le pre-condizioni devono essere soddisfatte prima dell'esecuzione del programma
  - le post-condizioni devono essere soddisfatte una volta terminata l'esecuzione del programma

{Pre}

Prog

{Post}

Es.18 Specifica di pre- e post-condizioni

Specificare pre e post-condizioni di un programma che, dati due interi x, y diversi da 0, restituisce $z = 1$ se x e y sono coprimi, $z = 0$ altrimenti.

Es.18

Specificare pre e post-condizioni di un programma che, dati due interi x, y diversi da 0, restituisce $z = 1$ se x e y sono coprimi, $z = 0$ altrimenti.

$$coprimi(x, y) \coloneqq \neg\exists k, n_1, n_2 (k \neq 1 \wedge kn_1 = x \wedge kn_2 = y)$$

Pre: $\{x > 0 \wedge y > 0\}$

Post: $\{(coprimi(x, y) \wedge z = 1) \vee (\neg coprimi(x, y) \wedge z = 0)\}$

Es.19 Specifica di pre- e post-condizioni

Specificare pre e post-condizioni di un programma che, dati tre interi x, y, z, restituisce $w = 1$ se uno dei tre è il prodotto degli altri due, $w = 0$ altrimenti.

Es.19

Specificare pre e post-condizioni di un programma che, dati tre interi x, y, z, restituisce $w = 1$ se uno dei tre è il prodotto degli altri due, $w = 0$ altrimenti.

Pre: $\{True\}$

Post: $\left\{ \begin{array}{c} \big(w = 1 \wedge (x = yz \vee y = xz \vee z = xy)\big) \\ \vee \\ \big(w = 0 \wedge (x \neq yz \wedge y \neq xz \wedge z \neq xy)\big) \end{array} \right\}$

Es.20 Specifica di pre- e post-condizioni

Specificare pre e post-condizioni di un programma che:

- Prende come input un array di lunghezza $NMAX$ le cui celle contengono interi non nulli fino a un certo indice, e le successive celle contengono 0 (cioè $\exists 0 \leq n < NMAX$ t.c. $A[0] \neq 0, \dots, A[n-1] \neq 0, A[n] = \cdots = A[NMAX-1] = 0$)
- Restituisce il numero n delle celle non nulle di A ed un array B di lunghezza $NMAX$ che sia una versione ordinata di A e senza ripetizioni (tranne gli 0). Anche in B le celle inutilizzate contengono 0.
- Si assuma di non dover controllare la lunghezza di A e B.

Ad esempio: sia $NMAX = 6$

- Input $A = [2,1,5,2,0,0]$
- Output $n = 4, B = [1,2,5,0,0,0]$

Es.20

Pre: $\left\{\exists n\left(0 \leq n \leq NMAX \wedge \forall i\left((i \geq n \rightarrow A[i] = 0) \wedge (\,0 \leq i < n \rightarrow A[i] > 0)\right)\right)\right\}$

Post: $\left\{\exists m \left\langle \begin{array}{c} 0 \leq m \leq n \\ \wedge \\ \forall j(j \geq m \rightarrow B[j] = 0) \\ \wedge \\ \forall j\ (0 \leq j < m - 1 \rightarrow B[j] < B[j+1]) \\ \wedge \\ \forall j\left(0 \leq j < m \rightarrow \exists i(0 \leq i < n \wedge A[i] = B[j])\right) \\ \wedge \\ \forall i\left(0 \leq i < n \rightarrow \exists j(0 \leq j < m \wedge A[i] = B[j])\right) \end{array} \right\rangle \right\}$

Es.21 Specifica di pre- e post-condizioni

Considerare la funzione $SPLIT$, che prende come input un array P (non vuoto) di interi ed un intero positivo e, e restituisce tre array (eventualmente vuoti) A, B, C tali che:

- l'array A contiene tutti gli elementi prima della prima occorrenza di e in P;
- l'array B contiene tutti gli elementi dopo la prima e prima dell'ultima occorrenza di e in P;
- l'array C contiene tutti gli elementi dopo l'ultima occorrenza di e in P.

Tutti gli elementi dell'array P (tranne la prima e l'ultima occorrenza di e) devono apparire in A, B, C preservando il loro ordine originale. Se e non occorre in P, allora $SPLIT$ deve restituire $B = P$ ed $A = C = \emptyset$.

Es.21

Ad esempio, dato $P = [3,0,2,1,2,0,2,1]$, A, B, C saranno:

- per $e = 1$, $A = [3,0,2]$, $B = [2,0,2]$, $C = [\ \ ]$
- per $e = 2$, $A = [3,0]$, $B = [1,2,0]$, $C = [1]$
- per $e = 3$, $A = [\ \ ]$, $B = [\ \ ]$, $C = [0,2,1,2,0,2,1]$
- per $e = 4$, $A = [\ \ ]$, $B = [3,0,2,1,2,0,2,1]$, $C = [\ \ ]$

Supponendo che l'indicizzazione degli array inizi da 0, e che ogni array X abbia un attributo $X.length$ che ne fornisce la lunghezza:

- Specificare le funzioni $first(X, a)$ e $last(X, a)$ che, dato un array X ed un intero a, restituiscono rispettivamente il minimo ed il massimo indice i tale che $X[i] = a$.
- Attenzione al caso in cui a non compaia in X!
- Specificare la funzione $SPLIT$ in termini di pre e post-condizioni, utilizzando le funzioni $first$ e $last$.

Es.21

$$first(X, a) = index \leftrightarrow$$
$$\big(0 \leq index < X.length \wedge X[index] = a \wedge \forall j(0 \leq j < index \rightarrow X[j] \neq a)\big)$$
$$\vee\ index = -1$$

$$last(X, a) = index \leftrightarrow$$
$$\big(0 \leq index < X.length \wedge X[index] = a \wedge \forall j(index < j < P.length \rightarrow X[j] \neq a)\big)$$
$$\vee\ index = X.length$$

Pre: $\{P.length > 0 \wedge e > 0\}$

Prog: $A, B, C = SPLIT(P, e)$

Post:
$$\left\{ \begin{array}{c} \forall i \left\langle \begin{array}{c} 0 \leq i < first(P,e) \rightarrow A[i] = P[i] \\ \wedge \\ first(P,e) < i < last(P,e) \rightarrow B[i - first(P,e) - 1] = P[i] \\ \wedge \\ last(P,e) < i < P.length \rightarrow C[i - last(P,e) - 1] = P[i] \end{array} \right\rangle \\ \wedge \\ A.length = first(P,e) \wedge B.length = last(P,e) - first(P,e) - 1 \wedge \\ C.length = P.length - last(P,e) - 1 \end{array} \right\}$$

Es.21

E se volessimo che, nel caso in cui e non occorra in P, la funzione $SPLIT$ restituisca $A = P, B = \emptyset, C = \emptyset$?

Basta modificare leggermente la funzione $first(X, a)$:

$$first'(X, a) = index \leftrightarrow$$
$$\big(0 \leq index < X.length \wedge X[index] = a \wedge \forall j(0 \leq j < index \rightarrow X[j] \neq a)\big)$$
$$\vee\ index = X.length$$